

```
Консоль отладки Microsoft Visual Studio

Демонстрация проверки валидности выдачи:
Валидность выдачи: True
Событие: Читатель Иван Иванов взял предмет с ID: a82b9ad3-1eb4-49e7-9fc1-3eff5bd6907a

Список всех книг:
Книга: Война и мир (Автор: Лев Толстой, ISBN: 978-5-389-06256-6, Жанр: Роман)
Книга: Катерина (Автор: Тарас Шевченко, ISBN: 978-5-389-06257-3, Жанр: Роман)

Список всех читателей:
Читатель: Иван Иванов, Email: ivan@example.com
Читатель: Петр Петров, Email: petr@example.com

Список всех записей выдачи:
Запись выдачи: ID=7503f28c-fdeb-4a76-b1c3-e53bbe334c64, Читатель=8ac2c623-6589-45c4-bb3a-c0d55bc26ee2, Предмет=a82b9ad3-1eb4-49e7-9fc1-3eff5bd6907a, Дата выдачи=15.05.2025 22:28:39, Возвращен=False, Дата возврата=Не возвращен
EventHandler: Статус книги Война и мир изменен на недоступна
Событие: Читатель Иван Иванов вернул предмет с ID: a82b9ad3-1eb4-49e7-9fc1-3eff5bd6907a

Данные после загрузки из файла:
Книга: Война и мир (Автор: Лев Толстой, ISBN: 978-5-389-06256-6, Жанр: Роман)
Книга: Катерина (Автор: Тарас Шевченко, ISBN: 978-5-389-06257-3, Жанр: Роман)

Нажмите любую клавишу для выхода...

C:\Users\Shiro\Desktop\Новая папка (4)\лаба2\Library.Console\bin\Debug\net8.0\Library.Console.exe (процесс 17604) завершил работу с кодом 0.
Чтобы автоматически закрывать консоль при остановке отладки, включите параметр "Сервис" -> "Параметры" -> "Отладка" -> "Автоматически закрыть консоль при остановке отладки".
Нажмите любую клавишу, чтобы закрыть это окно:
```

Интерфейс ICrudServiceAsync:

- Це асинхронний інтерфейс для CRUD-операцій
- Використовує generic-тип T для забезпечення типобезпечності
- Дозволяє працювати з різними типами даних без дублювання коду
- У нашому проєкті ми використовуємо його для роботи з Book, Reader і BorrowingRecord

Процес і потік:

- Процес — це екземпляр програми з власним адресним простором
- Потік — це одиниця виконання всередині процесу
- Багатопотоковість — це здатність програми виконувати кілька потоків одночасно
- У нашому проєкті ми можемо використовувати багатопотоковість для паралельної обробки запитів

Асинхронність і багатопотоковість:

- Асинхронність — це спосіб виконання операцій без блокування основного потоку
- Багатопотоковість — це паралельне виконання кількох потоків
- Асинхронність не обов'язково означає багатопотоковість
- У нашому проєкті ми можемо використовувати асинхронність для операцій введення/виведення (збереження/завантаження файлів)

async/await:

- async — вказує, що метод є асинхронним
- await — очікує завершення асинхронної операції
- Дозволяє писати асинхронний код у синхронному стилі
- У нашому проєкті ми можемо використовувати їх для асинхронних операцій із файлами

Task vs ValueTask:

- Task — тип посилення для асинхронних операцій
- ValueTask — значущий тип, ефективніший для коротких операцій
- ValueTask краще використовувати, коли операція часто завершується синхронно
- У нашому проєкті ми використовуємо Task для асинхронних операцій

Потокобезпечні колекції:

Колекції, безпечні для використання в багатопотоковому середовищі

Приклади: ConcurrentDictionary, ConcurrentQueue, ConcurrentBag

У нашому проєкті ми можемо використовувати їх для безпечного доступу до даних

Примітиви синхронізації:

lock — блокує доступ до ресурсу

Semaphore — обмежує кількість потоків

AutoResetEvent — сигналізує про завершення операції

У нашому проєкті ми можемо використовувати їх для синхронізації доступу до файлів

Безпека при одночасному доступі:

Використання потокобезпечних колекцій

Застосування примітивів синхронізації

Реалізація патерну "Монітор"

У нашому проєкті ми можемо використовувати lock для операцій із файлами

LINQ для статистики:

```
var minYear = books.Min(b => b.Year);
```

```
var maxYear = books.Max(b => b.Year);
```

```
var avgYear = books.Average(b => b.Year);
```

Різниця між методами LINQ:

Select — перетворює елементи

Where — фільтрує елементи

Aggregate — застосовує функцію до елементів

OrderBy — сортує елементи

Переваги LINQ:

Декларативний стиль

Відкладене виконання

Типобезпечність

Читабельність коду

Паралельне збереження:

Може призвести до пошкодження файлу

Потрібно використовувати синхронізацію

У нашому проєкті ми використовуємо lock для захисту операцій із файлами

Parallel.For:

Паралельне виконання циклів

Краще використовувати для CPU-інтенсивних операцій

У нашому проєкті можна використовувати для паралельної обробки даних

Пагінація:

Розбиття даних на сторінки

У нашому проєкті можна реалізувати так:

```
public IEnumerable<T> GetPage(int pageNumber, int pageSize)
{
    return _items.Skip((pageNumber - 1) * pageSize).Take(pageSize);
}
```